

Reviewer Pack — Irreducible Component Count Bounds Circuit Complexity for CN...

Entry 1f65a5eb95d6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 02:01:16 UTC

Irreducible Component Count Bounds Circuit Complexity for CNF

Entry ID: 1f65a5eb95d6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 02:01:16 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry **Field B** (complexity object): Boolean Circuits

Statement:

For a random 3-CNF formula F with n variables, the number of irreducible components of the variety defined by its clauses is $\Theta(\log s(F))$, where $s(F)$ is the minimal circuit size computing F .

Rationale (proposer's reasoning):

Irreducible components capture structural complexity of polynomial systems; their count may reflect the 'geometric complexity' of solving F , linking algebraic geometry to circuit size via the Nullstellensatz's degree bounds.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 09846e05980412dc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_3cnf(n: int, m: int) -> list:
    clauses = []
    for _ in range(m):
        literals = [random.choice([1, -1]) * (i + 1) for i in random.sample(range(n), 3)]
        clauses.append(literals)
    return clauses

def gcd(a: int, b: int) -> int:
    while b:
        a, b = b, a % b
    return a

def lcm(a: int, b: int) -> int:
    return abs(a * b) // gcd(a, b)

def extended_gcd(a: int, b: int) -> tuple:
    if a == 0:
        return (b, 0, 1)
    else:
        g, x, y = extended_gcd(b % a, a)
        return (g, y - (b // a) * x, x)

def mod_inverse(a: int, m: int) -> int:
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError('Modular inverse does not exist')
    else:
        return x % m

def matrix_mod_inv(matrix: list, m: int) -> list:
    n = len(matrix)
    augmented_matrix = [[matrix[i][j] for j in range(n)] + [0 if i != j else 1 for j in range(n)]]
    for i in range(n):
        pivot = augmented_matrix[i][i]
        for j in range(i, n * 2):
            augmented_matrix[i][j] //= pivot
        for k in range(n):
            if k != i:
                factor = augmented_matrix[k][i]
                for j in range(i, n * 2):
                    augmented_matrix[k][j] -= factor * augmented_matrix[i][j]
    return [[augmented_matrix[i][j] for j in range(n, 2 * n)] for i in range(n)]
```

```

def matrix_mod_mul(A: list, B: list, m: int) -> list:
    n = len(A)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] += A[i][k] * B[k][j]
            result[i][j] %= m
    return result

def matrix_mod_pow(matrix: list, exp: int, m: int) -> list:
    n = len(matrix)
    result = [[0 if i != j else 1 for j in range(n)] for i in range(n)]
    base = matrix
    while exp > 0:
        if exp % 2 == 1:
            result = matrix_mod_mul(result, base, m)
            base = matrix_mod_mul(base, base, m)
            exp //= 2
    return result

def gaussian_elimination(matrix: list) -> list:
    n = len(matrix)
    for i in range(n):
        pivot_row = i
        for j in range(i + 1, n):
            if abs(matrix[j][i]) > abs(matrix[pivot_row][i]):
                pivot_row = j
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        for j in range(n):
            if j != i:
                factor = matrix[j][i] // matrix[i][i]
                for k in range(n + 1):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def rank(matrix: list) -> int:
    n = len(matrix)
    rref = gaussian_elimination(matrix)
    return sum(1 for row in rref if any(row))

def irreducible_components(clauses: list, n: int) -> int:
    m = len(clauses)
    A = [[0] * (n + 2) for _ in range(n + 2)]
    for i in range(m):
        for j in range(3):
            x = abs(clauses[i][j]) - 1
            if clauses[i][j] > 0:
                A[x][x + n + 1] += 1
            else:
                A[n + 1][x] += 1
    A[n + 1][n + 1] = m
    rank_A = rank(A)
    return n - rank_A

```

```

def sat_solver_lower_bound(clauses: list) -> int:
    # Placeholder for SAT solver lower bound calculation
    # This is a simplified example and may not be accurate
    return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    m = 10 * n
    clauses = generate_3cnf(n, m)
    component_count = irreducible_components(clauses, n)
    s_F = sat_solver_lower_bound(clauses)
    log_s_F = math.log2(s_F) if s_F > 0 else -math.inf
    return {
        "metric_name": "component_count",
        "metric_value": component_count,
        "instances_tested": 1,
        "conjecture_holds": abs(component_count - log_s_F) < 1e-2,
        "counterexample": "" if conjecture_holds else f"component_count={component_count}, log(s(F))={log_s_F}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{results[0]['counterexample']}'\n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6a3bc472.py", line 142, in <module>
    result = run_trial(seed)
    ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6a3bc472.py", line 126, in run_trial
    component_count = irreducible_components(clauses, n)

```

